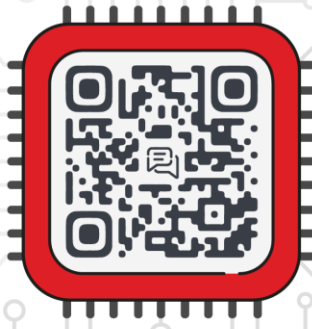




نظري

2

8

1200 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Construction & Deployment

د. سعيد أبو طراب



RB Informatics; 15/04/2026

محتوى مجاني غير مخصص للبيع التجاري

هندسة برمجيات ٢

مقدمة: تحدثنا في مقرر هندسة البرمجيات 1 عن مفهوم SDLC (Software development life cycle) وهي المراحل التي تؤمن الحصول على منتج برمجي ضمن مواصفات معينة وزمن معين. تحدثنا عن Communication, Planning, Modeling وفي هذه المحاضرة سنتحدث عن construction and Deployment.

Construction

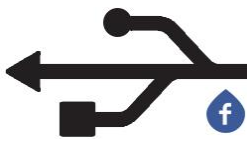
- تعد مرحلة construction إحدى المراحل الأساسية في دورة حياة تطوير البرمجيات (SDLC)، وهي المرحلة التي تلي التصميم مباشرة، الهدف منها هو تحويل المخططات والتصاميم (Design) التي تم اعتمادها في المراحل السابقة إلى منتج برمجي أو كود قابل للتشغيل.
- الهدف الفعلي هو أن نصل لقناعة بأن مرحلة ال construction ليست فقط بناء كود، بل هي أكبر بكثير، حيث أننا بحاجة لتحضير tasks كاملة وفهم كامل لمتى يجب علينا كتابة هذا الكود ومتى يجب أن نتواصل مع system خارجي مثلاً، إذاً علينا أن نقوم ببناء tasks حقيقية.
- عملية بناء الكود تحتاج إلى فريق على عكس المراحل السابقة وهذا ضروري ليكون العمل متناسقاً بينهم ويكونوا قادرين على تحديد tasks ، ومعرفة التكامل بينهم ليكون العمل صحيحاً.

هل عملية construction بحاجة إلى Planning؟

- تحدثنا بمفهوم ال SDLC أن ال Planning له مرحلة مفصلة ومستقلة بذاتها، لكننا لا نستطيع أن نقوم بتحديد تفصيلي لكل شيء نريد فعله من مرحلة ال planning، يبقى هناك ثغرات بالمعرفة وثغرات بالمعلومات والقرارات نأخذها ببداية كل مرحلة أو قرارات سوف نقوم باتخاذها بشكل مباشر.
 - فالخطة التفصيلية في مرحلة construction نشقها من الخطة العامة الموضوعة في مرحلة ال Planning.
- تساهم هذه المرحلة لمجموعة من القرارات والتقنيات التي تضمن سير العمل بكفاءة وتشمل:

1. Selection :

اختيار الأدوات التي سوف نعمل عليها والمقايير مسبقة الصنع أو مسبقة التحضير وتشمل:



1. Framework والتي تتطلب:

- Language
- IDE
- DBM (Database management system)
- API Protocol

2. 3parties

هي مكونات برمجية جاهزة أو خدمات خارجية يتم اختيارها وتجهيز البيئة للتخاطب معها، تستخدم لتحقيق التكامل (Integration) مع External System، مما يوفر الوقت والجهد في بناء وظائف معقدة موجودة مسبقاً لدى مزودين آخرين.

مثال: Google Services ، Amazon Services.

3. packages tools libraries

4. reused component

إمكانية إعادة استخدام مكونات برمجية جاهزة لتوفير الجهد وهو ما يتماشى مع مبدأ "عدم إعادة اختراع العجلة" في هندسة البرمجيات.

فمعرفة الأدوات والمصادر التي سوف نقوم باستخدامها هي الخطوة الأولى التي نقوم بها في مرحلة construction.

2. Coding:

أولى المشاكل التي تواجهنا في هذه المرحلة هي:

1. تقسيم العمل

2. ومن أين سوف نبدأ.

ولا تقتصر هذه المرحلة على كتابة التعليمات البرمجية فقط، بل تشمل ممارسات هندسية دقيقة، منها:

- Refactoring:** هي عملية إعادة هيكلة الكود بهدف تحسين بنيته وقراءته دون تغيير وظائفه الأساسية، وهي تعد عملية ضرورية وليست ترفيفية
- code style:** مجموعة من القواعد مثل توحيد تسمية المتغيرات وطريقة كتابة التعليقات مما يسهل على أي مطور آخر فهم الكود.
- بناء قواعد البيانات (Database Implementation):** خلق الجداول والعلاقات بينها بناءً على مخطط الـ ERD.
- UI/UX:** وهي مرحلة تصميم واجهات المستخدم (الهوية البصرية) وهذه المرحلة مختلف عليها أي من الممكن القيام بها بشكل كامل خلال مرحلة Design أو قد يتم تقسيمها على مرحلتين بحيث يتم إنجاز قسم الـ wireframe التي تحوي فقط تفاصيل الواجهات في مرحلة الـ Design وبعد ذلك في مرحلة الـ construction يتم بناء الـ UI/UX النهائي

وهي عملية الربط بين حلقات الـ coding المختلفة.

Source Code management System & Task management System

إن الـ Task management System تكون موجودة معنا في كافة مراحل المشروع ونستفيد

منها في توزيع الـ tasks، ولكن، هل تكفي لوحدها؟

- لا، يجب أن يكون هناك Integration بين الأجزاء والتأكد أن الكود المصدري محفوظ ومنظم وهذا ما نسميه بـ Source code management system، إذ يلزمنا أدوات مختصة بإدارة الأكواد، توزيع الـ tasks، ضبط الـ branches، ضبط المخرجات، وهذا ما نسميه بالـ Git الذي يدعم:

1. centralized management

2. version management

وجه المقارنة	Task management System	Source code management system
التركيز	تنظيم العمل وتوزيع المهام بين الأشخاص	إدارة الملفات البرمجية وضمان تكاملها
الوظيفة التقنية	تفكيك المتطلبات (Tasks Breakdown)	إدارة الفروع - الدمج - التراجع (Rollback- Merge- Branches)
إدارة الوقت	ترتيب الأولويات (prioritization)	إدارة أرقام الإصدارات (versioning)
الأدوات	منصات إدارة المشاريع	Git- GitHub- GitLab

هل من الضروري العمل على منصات مثل: Github و Gitlab من أجل الاستفادة من خدمات Git أو العمل عليه بشكل خاص؟

لا ليس من الضروري العمل على منصات مثل Github فمن الممكن أن يكون لدي server خاص بي يدعى server development خاص لعمليات التطوير ومن ثم أقوم بوضع جميع تعليمات Git وبذلك أضمن أن الـ source code لدي. وهذا ما يستخدم في أغلب الشركات المتخصصة بحيث تقوم بعمل بيئة خاصة بها بالإضافة إلى setup و configuration.



كل يوم هو فرصة جديدة لتكون أفضل نسخة من نفسك..

يُعطى لكل إصدار رقم يتبع صيغة (X. X. X. X) حيث:

1. الرقم الأول (من اليسار): Major change عند حدوث تغيير جذري.
2. الرقم الثاني: Release number عند إضافة أو تحديث ميزات بناءً على الـ Feedback.
3. الرقم الثالث: Error correction لتصحيح خطأ معين.
4. الرقم الرابع: Building number يزداد هذا الرقم تلقائياً مع كل عملية بناء أو تكامل (Integration) يقوم بها الفريق البرمجي. تكمن الفائدة الأساسية منه في تتبع التعديلات بدقة؛ فعند استلام ملاحظات (Feedback) حول خطأ ما، يمكن للمطورين تحديد "رقم البناء" الدقيق الذي حدث فيه الخطأ للتعامل معه قبل طرح النسخة للمستخدمين (Release) ومن الناحية التنظيمية، يُعتبر هذا الرقم داخلياً (Internal) في أغلب الأحيان، حيث لا يظهر للمستخدم النهائي ويبقى مستخدماً فقط داخل أروقة الشركة البرمجية لضبط الجودة.

❖ **منهجيات تنفيذ مهام البناء (construction Strategies):**

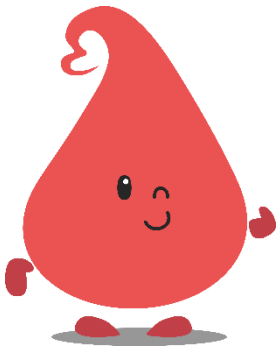
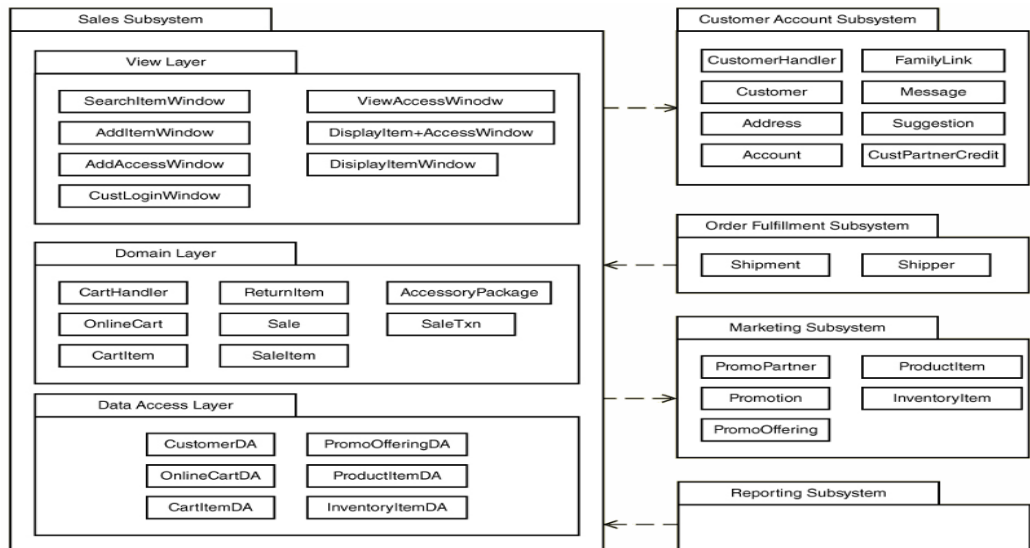
هناك عدة نماذج لتنفيذ المهام البرمجية لكل منها إيجابيات وسلبيات.

1. IPO (Input Process output) يتكون من ثلاث مراحل:

- المدخلات (Input): هي البيانات التي يتم إدخالها في النظام.
- المعالجة (Process): هي العمليات التي يتم تنفيذها على المدخلات.
- الإخراج (Output): هي النتائج التي يتم إنتاجها من المعالجة.

مميزته تسهيل الاختبار المبكر للواجهات مع المستخدم.

- **فائدة:** يوجد نوع من المخططات لم نمرّ به مسبقاً يُدعى Package Diagram ، وهو مخصص لإظهار الاعتماديات (dependencies) في النظام الذي نعمل عليه، مما يمنحنا نظرة أوسع كما في هذا المثال. development Order نرى فيه الاعتماديات بين المكونات (components) ، ولا يُعد استخدامه ضرورياً إلا إذا احتجنا إليه في هذه المرحلة.



2. Top Down :

البدء بواجهات المستخدم (View Layer) ثم الانتقال للطبقات الأدنى.

3. Bottom-Up :

البدء من قاعدة البيانات (Data Access Layer) صعوداً للواجهات.

4. Use case Driven :

يتم البناء ميزة بميزة (Feature by Feature) بربط الواجهة والمنطق وقاعدة البيانات لكل وظيفة بشكل كامل.

ملاحظة: Sandwich هي عبارة عن طريقة تمثيل حيث يعمل ال Frontend و ال Backend سوياً بإحدى الطرق Bottom-Up / Top Down / IPO.

فائدة: عندما نبدأ بالواجهات نعطي easy Validation و easy Testing لا user أما عندما نبدأ ب Database تصبح عملية testing أصعب لأننا نحتاج إلى أداة اختبار، وصاحب القرار النهائي هو الزبون وهو من يحدد البيانات التي تخزن والبيانات التي هي فقط temporary.

Deployment

- تُعرف مرحلة deployment بأنها مجموعة الأنشطة المطلوبة لجعل النظام البرمجي الجديد قابلاً للتشغيل الفعلي (operational).
- تبدأ هذه المرحلة عندما يصبح التطبيق متاحاً للنشر وتوفر التأكيدات المنطقية والواقعية بأنه يعمل بشكل سليم قبل إخرجه للمستخدم النهائي.
- تكمُن أهميتها في كونها الجسر الذي ينقل المنتج من بيئة التطوير إلى بيئة الإنتاج (Production Environment).

Deployment Activities ↩

1. Configuration Management :

تتضمن هذه المرحلة تجهيز البيئة الحاضنة ورفع التطبيق ليعمل بشكل نظامي، وتشمل الأنشطة التالية:

- Setup Environment
- Server Management
- Virtual Machines
- ربط النظام بنظام إدارة الكود (Git) وتفعيل (CI/CD)
- Continuous Integration / Continuous Deployment
- Technical Libraries

2. Data Migration:

تعتبر من أصعب المهام في مرحلة النشر، وهي عملية نقل البيانات من النظام القديم إلى النظام الجديد، وتشمل تحدياتها:

- اختلاف الهيكلية (Schema Mismatch): قد تكون قاعدة البيانات الجديدة تتبع هيكلية مختلفة تماماً عن القديمة أو تستخدم لغات ومحركات قواعد بيانات مختلفة (مثل الانتقال من Sql إلى Oracle).
- تنظيف البيانات (Data cleansing): التحقق من الشروط، وتعبئة البيانات الناقصة، ومنع التضاربات بين الحقول والجداول الجديدة والقديمة.
- إدخال البيانات (Data Entry): في حال عدم وجود نظام سابق (كان العمل ورقياً مثلاً)، يتم توظيف فريق لإدخال البيانات يدوياً أو استخدام نصوص برمجية (Scripts) مع ضرورة وجود نظام تحقق Validation.

3. Testing & Validation:

الاختبار في هذه المرحلة هو أعقد من الاختبار أثناء مرحلة التكويد لأنه يواجه:

- اختبار القبول (Acceptance Test): يتم على مستوى المستخدم للتأكد من أن النظام يلبي توقعاته.
- نسخة الألفا (Alpha Version): نسخة تجريبية غير مكتملة من النظام تهدف إلى اكتشاف الأخطاء وتصحيحها في مرحلة مبكرة قبل طرح نسخ أكثر استقراراً للمستخدمين الخارجيين.
- نسخة البيتا (Beta Version): إصدار نسخة تجريبية لمستخدمين حقيقيين للحصول على Feedback حول أداء النظام.
- اختبارات الجهد (Stress Tests): التأكد من قدرة النظام على تحمل أعداد كبيرة من المستخدمين والعمليات.

■ فائدة:

بعدما تعرفنا على نسخة الألفا والبيتا هناك أيضاً نسخة الإنتاج ونسخة الصيانة.
نسخة الإنتاج (Production Version) أو تسمى (Release version) أو (Production release):
نسخة النظام التي يتم توزيعها رسمياً على المستخدمين أو تشغيلها فعلياً للاستخدام طويل الأمد.
نسخة إصدار الصيانة (Maintenance Release): تحديث للنظام تهدف بشكل أساسي إلى تقديم إصلاحات للأخطاء (Bug fixes) وإجراء تغييرات طفيفة على الميزات الموجودة مسبقاً في البرمجيات.

4. Documentation & Training:

■ Documentation: ينقسم إلى نوعين أساسيين:

1. توثيق المستخدم (User Documentation): أدلة مساعدة وتعليمات حول كيفية استخدام النظام.
2. التوثيق التقني (System/Technical Documentation): يصف البنية المعمارية للنظام (Architecture) لتمكين المطورين من التعديل أو الصيانة مستقبلاً.

- Training of Trainers: بدلاً من تدريب كل موظف على حدة، يتم تدريب قسم المعلوماتية في المؤسسة والذي يتولى بدوره تدريب باقي الموظفين.

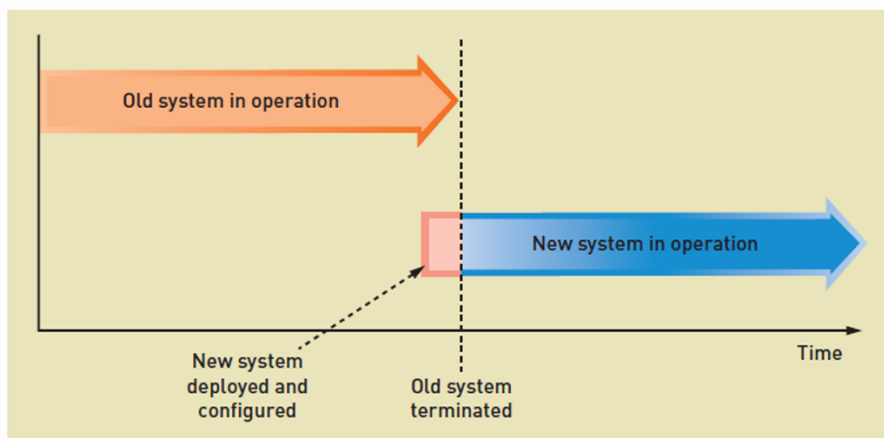
5. Deployment Strategies (استراتيجيات وآليات النشر):

توجد ثلاث طرق رئيسية لتنفيذ عملية النشر:

1. النشر المباشر (Direct Deployment):

إيقاف النظام القديم وتشغيل الجديد فوراً، ميزته البساطة وقلة التكلفة وعيبه الخطورة العالية لعدم وجود نظام بديل في حال حدوث فشل.

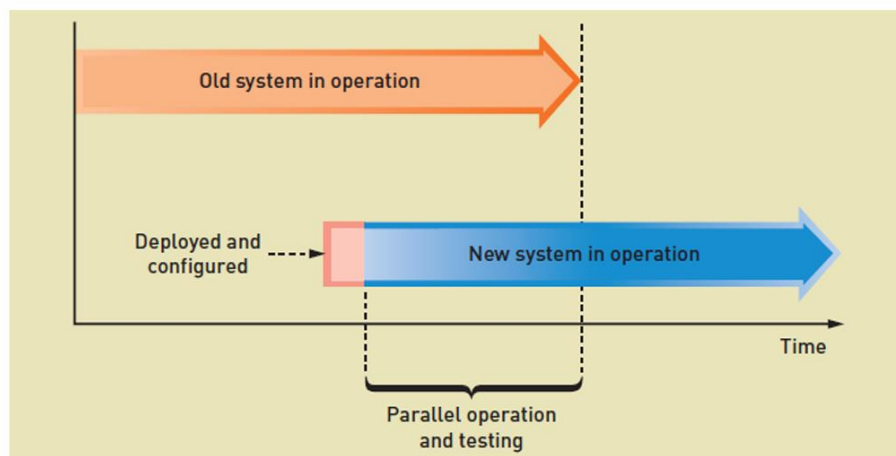
Direct Deployment



2. النشر المتوازي (Parallel Deployment):

تشغيل النظامين القديم والجديد معاً لفترة زمنية، ميزته تقليل المخاطر وعيبه التكلفة العالية وتضارب الموارد (Resources Conflict) وزيادة الجهد على الموظفين.

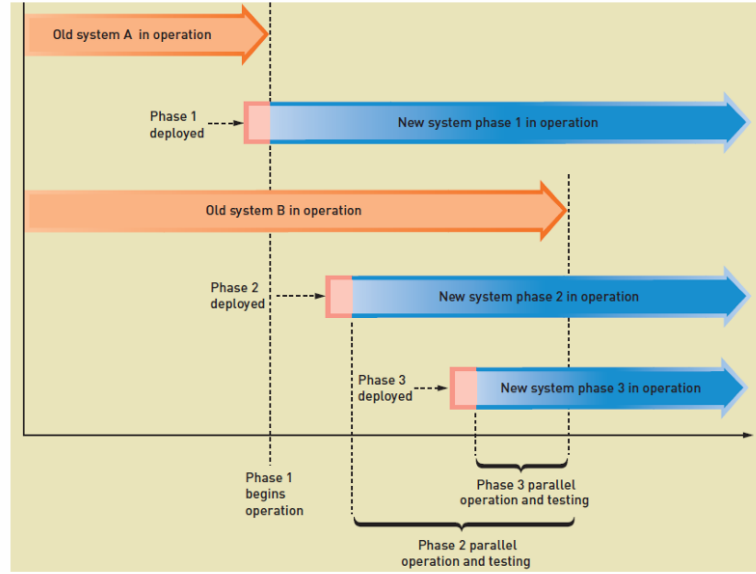
Parallel Deployment



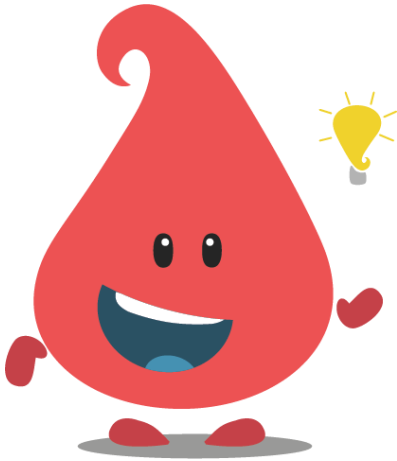
3. النشر المرحلي (Phased Deployment):

تنفيذ النظام على مراحل صغيرة أو "شرائح" وظيفية أو جغرافية، يقلل المخاطر بشكل كبير ولكنه يزيد من تعقيد العمل وصعوبة الفصل بين الأجزاء.

Phased Deployment



فائدة: هندسة البرمجيات هي مجموعة من المواقع وكل موقع له التبريرات الخاصة فيه، بما أنك تعلم لماذا اتخذت هذا القرار اتخذ هذا القرار بشكل صحيح.



انتهت المحاضرة